

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №1
на тему
**«СИММЕТРИЧНАЯ КРИПТОГРАФИЯ. СТАНДАРТ ШИФРОВАНИЯ
ГОСТ 28147-89»**

БГУИР 6-05-0612-02 098

Выполнил студент группы 353502
ПЕТРОЧЕНКО Михаил Максимович

(дата, подпись студента)

Проверил ассистент кафедры информатики
ГЕРЧИК Артём Вадимович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1	Цель работы	3
2	Ход работы	4
2.1	Режим простой замены	4
2.2	Режим гаммирования	4
2.3	Режим гаммирования с обратной связью	5
2.4	Генерация имитовставки	5
	Заключение	6
	Приложение А (обязательное) Листинг программного кода	7

1 ЦЕЛЬ РАБОТЫ

Целью данной лабораторной работы является изучение принципов построения симметричных блочных шифров на примере отечественного стандарта шифрования ГОСТ 28147-89. Задачи включают изучение структуры сети Фейстеля, лежащей в основе стандарта, проектирование и реализацию программного средства, осуществляющего шифрование и дешифрование данных в режимах простой замены, гаммирования и гаммирования с обратной связью, а также генерацию имитовставки и тестирование разработанного программного средства.

Конечным результатом данной лабораторной работы должна быть разработанная программа, осуществляющая шифрование и дешифрование данных по ГОСТ 28147-89 во всех перечисленных режимах и вычисляющая имитовставку заданной длины.

2 ХОД РАБОТЫ

В ходе выполнения работы было создано программное средство на языке программирования *Rust*. Реализация алгоритма ГОСТ 28147-89 вынесена в отдельный модуль `encryption::gost28147_89`, взаимодействие с пользователем осуществляется через текстовое меню, позволяющее задать открытый текст, 256-битный ключ и выбрать режим работы.

Алгоритм построен по схеме сети Фейстеля с 32 раундами преобразования над 64-битным блоком, разделённым на две 32-битные половины. Ключ имеет длину 256 бит и представлен восемью 32-битными подключами, порядок использования которых при зашифровании и расшифровании различается. Основная функция раунда выполняет сложение половины блока с подключом по модулю 2^{32} , замену по таблице S-блоков и циклический сдвиг влево на 11 бит.

Исходный код разработанного программного средства приведён в приложении А: реализация алгоритма представлена в листинге А.1, точка входа в программу – в листинге А.2.

2.1 Режим простой замены

В режиме простой замены каждый 64-битный блок открытого текста преобразуется независимо от остальных, что не требует синхропосылки, но сохраняет одинаковые блоки шифротекста для одинаковых блоков открытого текста. Результат работы программы в данном режиме представлен на рисунке 2.1.

```
plaintext : AAAAAAAAAAAAAAAAAAAAAA (25 bytes)
encrypted : 3D75C120DCE30E2F3D75C120DCE30E2F3D75C120DCE30E2FC694478853AAE0D407 (33 bytes)
decrypted : AAAAAAAAAAAAAAAAAAAAAA (25 bytes)
```

Рисунок 2.1 – Шифрование в режиме простой замены

2.2 Режим гаммирования

В режиме гаммирования шифрование выполняется наложением на открытый текст гаммы, вырабатываемой из синхропосылки при помощи констант C_1 и C_2 . Данный режим не требует дополнения сообщения до целого числа блоков и исключает совпадение блоков шифротекста. Результат работы программы в данном режиме представлен на рисунке 2.2.

```
plaintext : AAAAAAAAAAAAAAAAAAAAAA (25 bytes)
encrypted : 2329D1E3FAE42A0FA85FEE7EA0D81F4E1E93955633EAA4D891B1399ADA26FE7F5C (33 bytes)
decrypted : AAAAAAAAAAAAAAAAAAAAAA (25 bytes)
```

Рисунок 2.2 – Шифрование в режиме гаммирования

2.3 Режим гаммирования с обратной связью

В режиме гаммирования с обратной связью очередной блок гаммы вырабатывается из предыдущего блока шифротекста, благодаря чему искажение одного блока влияет на результат расшифрования последующих. Результат работы программы в данном режиме представлен на рисунке 2.3.

```
plaintext : AAAAAAAAAAAAAAAAAAAAAA (25 bytes)
encrypted : F39FDD3F5E183C9D4072F9235ECDA05D0133B8621F8CE11C4072F9235ECDA05D01 (33 bytes)
decrypted : AAAAAAAAAAAAAAAAAAAAAA (25 bytes)
```

Рисунок 2.3 – Шифрование в режиме гаммирования с обратной связью

2.4 Генерация имитовставки

Имитовставка генерируется в режиме, использующем 16 раундов преобразования вместо 32, и предназначена для контроля целостности передаваемого сообщения. Длина имитовставки задаётся пользователем и не превышает 32 бит. Результат работы программы представлен на рисунке 2.4.

```
MAC length in bits [32]: 32
message : AAAAAAAAAAAAAAAAAAAAAA (25 bytes)
mac      : 42E0E3E6 (32 bits)
```

Рисунок 2.4 – Генерация имитовставки

Таким образом все режимы работы описываемого стандартом ГОСТ 28147-89 алгоритма были изучены, реализованы и протестированны в полном объёме.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены принципы построения симметричных блочных шифров на примере алгоритма ГОСТ 28147-89. Было разработано программное средство на языке *Rust*, реализующее шифрование и дешифрование данных в режимах простой замены, гаммирования и гаммирования с обратной связью, а также генерацию имитовставки заданной длины.

Тестирование показало, что расшифрование во всех реализованных режимах восстанавливает исходное сообщение без искажений, что подтверждает корректность реализации алгоритма. Сравнение режимов показало, что режим простой замены сохраняет структуру открытого текста и потому непригоден для шифрования сообщений, содержащих повторяющиеся блоки, тогда как режимы гаммирования лишены данного недостатка.

Все задачи лабораторной работы выполнены в полном объёме.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг программного кода

Листинг А.1 – Реализация алгоритма ГОСТ 28147-89 на *Rust*

```
use std::{fmt::Display, ops::Index, vec};

use rand::RngExt;

const BLOCK_SIZE: usize = 8;
const MAC_ITERATIONS: usize = 16;
pub const MAX_MAC_BITS: usize = 32;
const ITERATIONS: usize = 32;
const S_BLOCKS: [[u32; 16]; 8] = [
    [4, 10, 9, 2, 13, 8, 0, 14, 6, 11, 1, 12, 7, 15, 5, 3],
    [14, 11, 4, 12, 6, 13, 15, 10, 2, 3, 8, 1, 0, 7, 5, 9],
    [5, 8, 1, 13, 10, 3, 4, 2, 14, 15, 12, 7, 6, 0, 9, 11],
    [7, 13, 10, 1, 0, 8, 9, 15, 14, 4, 6, 12, 11, 2, 5, 3],
    [6, 12, 7, 1, 5, 15, 13, 8, 4, 10, 9, 14, 0, 3, 11, 2],
    [4, 11, 10, 0, 7, 2, 1, 13, 3, 6, 8, 5, 9, 12, 15, 14],
    [13, 11, 4, 1, 3, 15, 5, 9, 0, 10, 14, 7, 6, 8, 2, 12],
    [1, 15, 13, 0, 5, 7, 10, 4, 9, 2, 3, 14, 6, 11, 8, 12],
];

const FOUR_BIT_MASKS: [u32; 8] = [
    0x0000000F, 0x000000F0, 0x0000F000, 0x0000F000, 0x000F0000, 0x00F00000,
    0x0F000000, 0xF0000000,
];

const C1: u32 = 0x01010104;
const C2: u32 = 0x01010101;

pub enum Gost28147_89Type {
    ECB,
    CTR,
    CFM,
}

pub struct Gost28147_89 {}

pub struct Gost28147_89Key(pub [u32; 8]);

impl Index<usize> for Gost28147_89Key {
    type Output = u32;

    fn index(&self, index: usize) -> &Self::Output {
        &self.0[index]
    }
}

impl Display for Gost28147_89Key {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        for &word in &self.0 {
            write!(f, "{:08X}", word)?;
        }

        Ok(())
    }
}

impl Gost28147_89 {
    fn get_subkey(key: &Gost28147_89Key, idx: usize, encrypt: bool) -> u32 {
```

```

        if idx > 31 {
            panic!("invalid index on K_{idx}")
        }
        if encrypt {
            if idx < 24 {
                return key[idx % 8];
            } else {
                return key[(32 - (idx + 1)) % 8];
            }
        } else {
            if idx < 8 {
                return key[idx];
            } else {
                return key[7 - idx % 8];
            }
        }
    }

    fn f(half: u32, subkey: u32) -> u32 {
        let sum = half.wrapping_add(subkey);
        let mut result: u32 = 0;
        for i in 0..8 {
            let four_bits = (sum & FOUR_BIT_MASKS[i]) >> (i * 4);
            let new_four_bits: u32 = S_BLOCKS[i][four_bits as usize] << (i *
4);

            result |= new_four_bits;
        }

        return result.rotate_left(11);
    }

    fn round(a: u32, b: u32, subkey: u32) -> (u32, u32) {
        return (b ^ Self::f(a, subkey), a);
    }

    fn encrypt_block(mut a: u32, mut b: u32, key: &Gost28147_89Key, encrypt:
bool) -> (u32, u32) {
        for iteration in 0..ITERATIONS {
            (a, b) = Self::round(a, b, Self::get_subkey(&key, iteration,
encrypt));
        }

        return (b, a);
    }

    fn mac_block(mut a: u32, mut b: u32, key: &Gost28147_89Key) -> (u32, u32)
{
        for iteration in 0..MAC_ITERATIONS {
            (a, b) = Self::round(a, b, Self::get_subkey(&key, iteration,
true));
        }

        return (a, b);
    }

    fn apply_algo(bytes: Vec<u8>, key: &Gost28147_89Key, encrypt: bool) ->
Vec<u8> {
        let blocks = bytes.chunks(BLOCK_SIZE);
        let mut result: Vec<u8> = vec![];

        for block in blocks {
            let a_bytes = block[0..(BLOCK_SIZE / 2)]

```



```

        .try_into()
        .expect("u32 can only be constructed from [u8;4]");
let b_bytes = block[(BLOCK_SIZE / 2)..]
        .try_into()
        .expect("u32 can only be constructed from [u8;4]");
let mut a = u32::from_ne_bytes(a_bytes);
let mut b = u32::from_ne_bytes(b_bytes);

(a, b) = Self::encrypt_block(a, b, key, encrypt);

result.extend_from_slice(&a.to_ne_bytes());
result.extend_from_slice(&b.to_ne_bytes());
}

return result;
}

pub fn encrypt_ecb(mut plain_bytes: Vec<u8>, key: &Gost28147_89Key) ->
Vec<u8> {
    let padding = {
        let pd = 8 - plain_bytes.len() % 8;
        if pd == 8 { 0 } else { pd }
    };
    for _ in 0..padding {
        plain_bytes.push(0);
    }

    let mut result = Self::apply_algo(plain_bytes, key, true);
    result.push(padding as u8);

    return result;
}

pub fn decrypt_ecb(mut encrypted_bytes: Vec<u8>, key: &Gost28147_89Key) -
> Vec<u8> {
    let padding = encrypted_bytes.pop().unwrap_or(0) as usize;

    let mut result = Self::apply_algo(encrypted_bytes, key, false);
    for _ in 0..padding {
        result.pop();
    }

    return result;
}

pub fn encrypt_ctr(plain_bytes: Vec<u8>, key: &Gost28147_89Key) ->
Vec<u8> {
    let mut result: Vec<u8> = vec![];
    let mut rng = rand::rng();
    let s_a: u32 = rng.random();
    let s_b: u32 = rng.random();

    result.extend_from_slice(&s_a.to_ne_bytes());
    result.extend_from_slice(&s_b.to_ne_bytes());

    let (mut n_1, mut n_2) = Self::encrypt_block(s_a, s_b, key, true);

    let blocks = plain_bytes.chunks(BLOCK_SIZE);
    for block in blocks {
        n_1 = n_1.wrapping_add(C1);
        n_2 = {
            let (sum, carry) = n_2.overflowing_add(C2);
            sum + carry as u32
        };
    };
}

```

```

        let (g_1, g_2) = Self::encrypt_block(n_1, n_2, key, true);

        let mut gamma: Vec<u8> = vec![];
        gamma.extend_from_slice(&g_1.to_ne_bytes());
        gamma.extend_from_slice(&g_2.to_ne_bytes());

        for (idx, byte) in block.iter().enumerate() {
            result.push(byte ^ gamma[idx]);
        }
    }

    return result;
}

pub fn decrypt_ctr(encrypted_bytes: Vec<u8>, key: &Gost28147_89Key) ->
Vec<u8> {
    let mut result: Vec<u8> = vec![];

    let s_a = u32::from_ne_bytes(
        encrypted_bytes[..4]
            .try_into()
            .expect("u32 can only be constructed from [u8;4]"),
    );
    let s_b = u32::from_ne_bytes(
        encrypted_bytes[4..8]
            .try_into()
            .expect("u32 can only be constructed from [u8;4]"),
    );

    let (mut n_1, mut n_2) = Self::encrypt_block(s_a, s_b, key, true);

    let blocks = encrypted_bytes[8..].chunks(BLOCK_SIZE);
    for block in blocks {
        n_1 = n_1.wrapping_add(C1);
        n_2 = {
            let (sum, carry) = n_2.overflowing_add(C2);
            sum + carry as u32
        };

        let (g_1, g_2) = Self::encrypt_block(n_1, n_2, key, true);

        let mut gamma: Vec<u8> = vec![];
        gamma.extend_from_slice(&g_1.to_ne_bytes());
        gamma.extend_from_slice(&g_2.to_ne_bytes());

        for (idx, byte) in block.iter().enumerate() {
            result.push(byte ^ gamma[idx]);
        }
    }

    return result;
}

pub fn encrypt_cfm(plain_bytes: Vec<u8>, key: &Gost28147_89Key) ->
Vec<u8> {
    let mut result: Vec<u8> = vec![];
    let mut rng = rand::rng();
    let s_a: u32 = rng.random();
    let s_b: u32 = rng.random();

    result.extend_from_slice(&s_a.to_ne_bytes());
    result.extend_from_slice(&s_b.to_ne_bytes());

```

```

    let (n_1, n_2) = Self::encrypt_block(s_a, s_b, key, true);
    let mut gamma: Vec<u8> = vec![];
    gamma.extend_from_slice(&n_1.to_ne_bytes());
    gamma.extend_from_slice(&n_2.to_ne_bytes());

    let blocks = plain_bytes.chunks(BLOCK_SIZE);
    for block in blocks {
        for (idx, byte) in block.iter().enumerate() {
            result.push(byte ^ gamma[idx]);
            let result_offset = result.len();
            gamma[idx] = result[result_offset - 1];
        }
    }

    return result;
}

pub fn decrypt_cfm(encrypted_bytes: Vec<u8>, key: &Gost28147_89Key) ->
Vec<u8> {
    let mut result: Vec<u8> = vec![];

    let s_a = u32::from_ne_bytes(
        encrypted_bytes[..4]
            .try_into()
            .expect("u32 can only be constructed from [u8;4]"),
    );
    let s_b = u32::from_ne_bytes(
        encrypted_bytes[4..8]
            .try_into()
            .expect("u32 can only be constructed from [u8;4]"),
    );

    let (n_1, n_2) = Self::encrypt_block(s_a, s_b, key, true);
    let mut gamma: Vec<u8> = vec![];
    gamma.extend_from_slice(&n_1.to_ne_bytes());
    gamma.extend_from_slice(&n_2.to_ne_bytes());

    let blocks = encrypted_bytes[8..].chunks(BLOCK_SIZE);
    for block in blocks {
        for (idx, byte) in block.iter().enumerate() {
            let byte_copy = byte;
            result.push(byte ^ gamma[idx]);
            gamma[idx] = *byte_copy;
        }
    }

    return result;
}

pub fn compute_mac(
    mut plain_bytes: Vec<u8>,
    key: &Gost28147_89Key,
    mac_bits: usize,
) -> Option<u32> {
    if mac_bits == 0 || mac_bits > MAX_MAC_BITS {
        panic!("mac bits must be in 1..={MAX_MAC_BITS}, got {mac_bits}")
    }
    if plain_bytes.is_empty() {
        return None;
    }

    let padding = {

```

```

        let pd = 8 - plain_bytes.len() % 8;
        if pd == 8 { 0 } else { pd }
    };
    for _ in 0..padding {
        plain_bytes.push(0);
    }
    let (mut s_1, mut s_2) = (0u32, 0u32);

    let blocks = plain_bytes.chunks(BLOCK_SIZE);
    for block in blocks {
        let (n_1, n_2) = (
            u32::from_ne_bytes(
                block[..(BLOCK_SIZE / 2)]
                    .try_into()
                    .expect("failed to compute mac"),
            ),
            u32::from_ne_bytes(
                block[(BLOCK_SIZE / 2)..]
                    .try_into()
                    .expect("failed to compute mac"),
            ),
        );

        (s_1, s_2) = Self::mac_block(n_1 ^ s_1, n_2 ^ s_2, key);
    }

    let mac_mask: u32 = u32::MAX >> (32 - mac_bits);

    return Some(s_2 & mac_mask);
}
}

```

Листинг А.2 – Точка входа в программу

```

mod encryption;
use encryption::gost28147_89::*;
use std::io::{self, Write};

const KEY: Gost28147_89Key = Gost28147_89Key([1, 2, 3, 4, 5, 6, 7, 8]);
const DEFAULT_MAC_BITS: usize = 32;
type CipherFn = fn(Vec<u8>, &Gost28147_89Key) -> Vec<u8>;

fn prompt(label: &str) -> String {
    print!("{label}");
    io::stdout().flush().expect("failed to flush stdout");

    let mut line = String::new();
    io::stdin()
        .read_line(&mut line)
        .expect("failed to read stdin");

    return line.trim_end_matches(['\r', '\n']).to_string();
}

fn to_hex(bytes: &[u8]) -> String {
    let mut hex = String::with_capacity(bytes.len() * 2);
    for byte in bytes {
        hex.push_str(&format!("{byte:02X}"));
    }

    return hex;
}

```

```

fn run_gost28147_89(plaintext: &str, key: &Gost28147_89Key, encryption_type:
Gost28147_89Type) {
    let (encryption_fn, decryption_fn): (CipherFn, CipherFn) = match
encryption_type {
        Gost28147_89Type::ECB => (Gost28147_89::encrypt_ecb,
Gost28147_89::decrypt_ecb),
        Gost28147_89Type::CTR => (Gost28147_89::encrypt_ctr,
Gost28147_89::decrypt_ctr),
        Gost28147_89Type::CFM => (Gost28147_89::encrypt_cfm,
Gost28147_89::decrypt_cfm),
    };

    let encrypted = encryption_fn(plaintext.as_bytes().to_vec(), key);
    let decrypted = decryption_fn(encrypted.clone(), key);

    println!("  plaintext : {plaintext} ({} bytes)", plaintext.len());
    println!(
        "  encrypted : {} ({} bytes)",
        to_hex(&encrypted),
        encrypted.len()
    );
    match String::from_utf8(decrypted) {
        Ok(text) => println!("  decrypted : {text} ({} bytes)", text.len()),
        Err(err) => println!("  decrypted : <not valid UTF-8> {err}"),
    }
}

fn run_mac(plaintext: &str, key: &Gost28147_89Key) {
    let bits_input = prompt(&format!("MAC length in bits
[DEFAULT_MAC_BITS]: "));
    let mac_bits = if bits_input.is_empty() {
        DEFAULT_MAC_BITS
    } else {
        match bits_input.parse::<usize>() {
            Ok(bits) if (1..=MAX_MAC_BITS).contains(&bits) => bits,
            _ => {
                println!("MAC length must be an integer in
1..={MAX_MAC_BITS}");
                return;
            }
        }
    };

    match Gost28147_89::compute_mac(plaintext.as_bytes().to_vec(), key,
mac_bits) {
        Some(mac) => {
            println!("  message : {plaintext} ({} bytes)", plaintext.len());
            println!("  mac      : {mac:08X} ({mac_bits} bits)");
        }
        None => println!("  an empty message has no MAC"),
    }
}

fn parse_key(hex_key: &str) -> Option<Gost28147_89Key> {
    if hex_key.len() != 64 {
        println!("valid key should be 256 bits (64 characters)");
        return None;
    }
    if !hex_key.chars().all(|c| c.is_ascii_hexdigit()) {
        println!("invalid hex");
        return None;
    }
}

```

```

let mut new_key = [0u32; 8];

for i in (0..64).step_by(8) {
    if let Ok(subkey) = u32::from_str_radix(&hex_key[i..i + 8], 16) {
        new_key[i / 8] = subkey;
    } else {
        unreachable!()
    }
}

Some(Gost28147_89Key(new_key))
}

fn main() {
    let mut plaintext = prompt("Plaintext: ");
    let mut key = KEY;

    loop {
        println!();
        println!("1) GOST 28147-89 (ECB)");
        println!("2) GOST 28147-89 (CTR)");
        println!("3) GOST 28147-89 (CFM)");
        println!("4) GOST 28147-89 (MAC)");

        let choice = prompt(
            "Choice (or 'p' to change the plaintext, 'q' to quit, 'k' to see
key, 'c' to change key): ",
        );
        println!();

        match choice.as_str() {
            "1" => run_gost28147_89(&plaintext, &key, Gost28147_89Type::ECB),
            "2" => run_gost28147_89(&plaintext, &key, Gost28147_89Type::CTR),
            "3" => run_gost28147_89(&plaintext, &key, Gost28147_89Type::CFM),
            "4" => run_mac(&plaintext, &key),
            "p" => plaintext = prompt("Plaintext: "),
            "q" => break,
            "c" => key = parse_key(prompt("Key: ").as_str()).unwrap_or(key),
            "k" => println!("Key is: {}", key),
            _ => println!("Unknown choice: {choice:?}"),
        }
    }
}

```